Political Narrative - Guidelines for Coding

1 Instructions to Adopt the Political Narrative Framework

We provide here a step-by-step explanation on how to adapt the code and run it on your machine.

- (For beginner Python users) The first step is to download Python at the link.
- (For beginner Python users) Then, we suggest you to download Anaconda and Anaconda Prompt at the link.
- Download the files in `code and prompts` and the `environment.yml`. Then, you must build the following folder structure locally:

```
<main>/
   code/
     annotation_openai_stage1.py
     annotation_openai_stage2.py
     prompts/
        system_message_stage1.json        # <-- your input message
        system_message_stage2.json        # <-- your input message
   data/
     output/
        your_data_stage1.csv              # <-- your input CSV
        your_data_stage2.csv              # <-- your input CSV
   output/
     data/
        openai_output/
        openai_final/
   logs/
   environment.yml                        # <-- your environment requiremnts
```

  According to this structure, you need to fill the `code/prompts/` folder with the system message of the stage, and the `data/output/` folder with the `.csv` dataset. Put the `environment.yml` in the main project folder.

  IMPORTANT: the input `.csv` must contain a column called `id` and a column called `text`

- Open Anaconda Prompt and move into the main project folder:

```
cd C:\Users\YourName\Documents\political_narrative_project
```

- Then, create the environment where the code will run:

```
conda env create -f environment.yml
```

- Activate the environment

```
conda activate political_narrative
```

- Install the correct version of `openai`

```
pip install "openai==2.4.0"
```

- Set the `OPENA_API_KEY` as an environmental variable in your current environment:

```
conda env config vars set OPENAI_API_KEY="sk-your-key-here"
conda deactivate
conda activate political_narrative
```

  This step is crucial for the success of the annotation process. This code requires an individual OpenAI API key, that the user can retrieve in his OpenAI personal page. This key is personal and directly linked to the user's wallet, so it's important to keep it personal and hidden in the machine and not in the script.

- Open Anaconda Prompt and activate Spyder or your preferred Python IDE by running the commands:

```
conda install spyder python=3.13.5
spyder
```

- Open the script of interest directly in spyder using the top left command bar. Here you can choose one of the two Python scripts depending on the task that you need to perform:
  - The stage 1 script `annotation_openai_stage1` allows to classify a text based on its relevance to the topic selected. This code returns an additional column to the input dataset that takes values from 0 to 3 (0 - irrelevant, 1 - assert, 2 - deny, 3 - relevant). This script is not strictly necessary for the character-roles annotation, but it can be useful to assess the relevance of a specific text to the topic at hand. For example, in *Gehring and Grigoletto (2025)* we use this script to filter the tweets and keep only those relevant to the topic of climate change policy.
  - The stage 2 script `annotation_openai_stage2` is the core of the Political Narrative Package, and it allows to retrieve the character-role classification. The code returns a dataset with a column for each specified character taking values from 0 to 4, where 0 is no-mention of the character, 1 is Villain role, 2 is Hero role, 3 is Victim role, and 4 for appearence of the character in none of these roles (Neutral).

- Regarding the prompt, you can adapt the prompt in the folder `code/prompts/` with your own instructions. You can do this by accessing the **"SYSTEM MESSAGE"** in the prompt. If you need more details about the prompt design, please go back to Step 4 of the online guide.

- Modify the input dataset: you need to provide in the `/data/output` folder your dataset named `your_data_stage2` (or `your_data_stage1`). This dataset must contain a column called `id` with the unique identifiers and a column called `text` containing the text for the classification. The .csv file must be UTF-8 encoded. If you see errors such `unicodeDecodeError` try saving again your file in UTF-8, or the script will automatically fall back to latin-1 encoding.

- Once prepared the folder structure, the prompt, and the dataset, you can make minimal changes to the Python script. First, changing the directory

```
main = r"D:\your directory"
```

  If you are running the `annotation_openai_stage1` code, no other changes are needed and you can directly run the script.

- Second, change the JSON structure based on the number of characters: depending on your character selection, you might need to modify the structure of the .JSON file that is created by OpenAI. Here, you just need to adapt the number of keys to your number of characters. You can access it by changing the properties in the following part of the script:

```
JSON_SCHEMA = {
 "name": "ArticleClassification",
 "schema": {
     "type": "object",
     "properties": {
        "items": {
            "type": "array",
            "items": {
                "type": "object",
                "properties": {
                    "id": {"type": ["integer", "string"]},
                    "a": {"type": "integer", "enum": [0, 1, 2, 3, 4]},
                    "b": {"type": "integer", "enum": [0, 1, 2, 3, 4]},
                    "c": {"type": "integer", "enum": [0, 1, 2, 3, 4]},
                    "d": {"type": "integer", "enum": [0, 1, 2, 3, 4]},
                    "e": {"type": "integer", "enum": [0, 1, 2, 3, 4]},
                    "f": {"type": "integer", "enum": [0, 1, 2, 3, 4]},
                    "g": {"type": "integer", "enum": [0, 1, 2, 3, 4]},
                    "h": {"type": "integer", "enum": [0, 1, 2, 3, 4]},
                    "i": {"type": "integer", "enum": [0, 1, 2, 3, 4]},
                    "j": {"type": "integer", "enum": [0, 1, 2, 3, 4]}
                    },
                "required": ["id","a","b","c","d","e","f", "g", "h", "i", "j"],
                "additionalProperties": False
            }
        }
     },
     "required": ["items"],
     "additionalProperties": False
},
"strict": True
}
```

- Then, you need to change the user content to adapt it to your number of characters, as done before:

```
def build_user_content(article_group):

    user_content = (
        "Classify the following opinion(s) strictly per the system instructions. "
        "Respond with **only** a JSON object of the form {\"items\": [ ... ]}, "
         "where \"items\" is an array of objects (one per text, same order). "
        "Each object must include keys: id, a, b, c, d, e, f, g, h, i, j (a{j in 0{4)
        "No extra text.\n\n"
     )
    for article in article_group:
        user_content += f"ID: {article['id']}\n"
         user_content += f"Text: {article['text']}\n"
    return user_content
```

- Lastly, you need to adapt the `flatten_results()` function by changing this few lines of code:

```
"request_id": result["request_id"],
"id": entry_id,
"text": result.get("article_text", None),
"a": entry.get("a", 0),
"b": entry.get("b", 0),
"c": entry.get("c", 0),
"d": entry.get("d", 0),
"e": entry.get("e", 0),
"f": entry.get("f", 0),
"g": entry.get("g", 0),
"h": entry.get("h", 0),
"i": entry.get("i", 0),
"j": entry.get("j", 0)
```

- It's finally time to run the code. You can do this either by highlighting the whole script and clickin `F9` on the keybord, or by clicking on the green triangle at the top of the Spyder interface. Your results are saved in `.csv` format as `flattened_results_all.csv`. These are stored in the local folder `main/output/data/openai_final/run20251027_133424/`, where the last part (`run20251027_133424`) takes his name from the date and time when the script has been compiled (date_time). This `.csv` file contains the results of the Openai classification.

Settings of the Model

Depending on the parameters, the annotation will return different results. In this package, you are allowed to change the temperature of the model by accessing the temperature field in the `send_request()` function:

```
def send_request(request_id, article_group, system_message):

    user_content = build_user_content(article_group)

    body = {
        "model": "gpt-4o",
        "messages": [
            {"role": "system", "content": system_message},
            {"role": "user", "content": user_content}
        ],
        "max_tokens": 300,
        "temperature": 0.5,   # <----- ACCESS IT FROM HERE
        "response_format": {
            "type": "json_schema",
            "json_schema": JSON_SCHEMA
        }
    }
```

Temperature is a parameter that controls the "creativity" or randomness of the text generated by the GPT model. A higher temperature (e.g., 0.7) results in more diverse and creative output, while a lower temperature (e.g., 0.2) makes the output more deterministic and focused. In practice, temperature affects the probability distribution over the possible tokens at each step of the generation process. A temperature of 0 would make the model completely deterministic, always choosing the most likely token. Lastly, the user can also decide wich GPT model to use. This code allows for gpt-4o, gpt-4o-mini, gpt-4.1, and gpt-4.1-mini.